
2D
ROGUELIKE
COMBAT
Blade and Bow
Unity

Overview

The story takes place in a fictional world with a medieval fantasy setting. The protagonist's main objective is to defeat enemies along the way and ultimately save the world. Players control a single character whose primary attack methods include sword strikes and shooting arrows. While avoiding traps, players must also focus on defeating enemies.

Character Design

The character has two attack modes: melee with a Blade and ranged attacks with a Bow.

The Blade is designed as a three-stage combo. This design reflects my intention to make combat the core gameplay element. A more diverse attack style provides players with a more satisfying combat experience, Additionally, I gave enemies a wider attack range, making it easier for players to get hit. This multi-stage attack system offers players more opportunities to counterattack.

In the second attack model, the bow will shoot arrows horizontally without any drop-off. This design aligns with the parkour elements in the game, where players must navigate obstacles while engaging in combat.





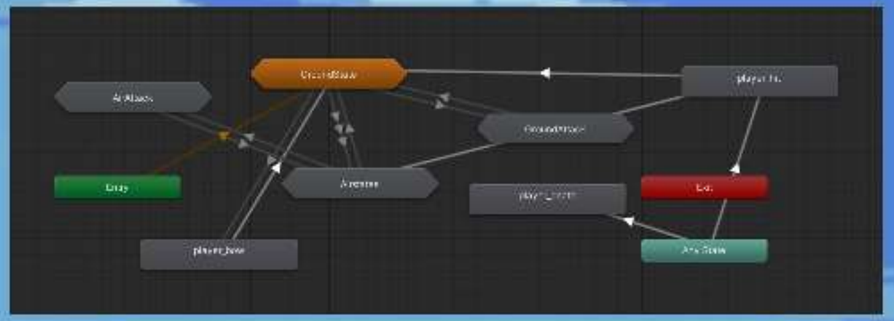
Level Mechanisms

Shattered Stone Slabs: These stepping stones are unstable and will break if the player stays on them for too long, causing them to fall into the abyss. Players must move quickly to avoid falling. At the same time, this design limits backtracking, confining the player to a specific area to enhance the intensity of combat.

In addition to basic melee enemies, the game features diverse level mechanics, such as the "Dragon" that swoops down in diving attacks. Its movements are fast, and it stays in the player's attack window for only a short time. Since the bow can only shoot horizontally, players are required to quickly change positions while attacking to effectively counter it.

Coins and health packs are common elements in platform games. They motivate players to explore further and provide timely replenishment. However, they can also serve as traps, luring players into challenging situations.

Code Logic



Character Animator State Machine

```
1 reference
private void Flight()
{
    Vector2 directionToWayPoint = (nextWayPoint.position - transform.position).normalized;
    float distance = Vector2.Distance(nextWayPoint.position, transform.position);
    rb.velocity = directionToWayPoint * flightSpeed;
    UpdateDirection();
    if (distance <= wayPointReachedDistance)
    {
        wayPointNum++;
        if (wayPointNum >= wayPoints.Count)
        {
            wayPointNum = 0;
        }
        nextWayPoint = wayPoints[wayPointNum];
    }
}
```

Flying Controller

```
0 references
public void OnRun(InputAction.CallbackContext context)
{
    if (context.started)
    {
        isRunning = true;
    }
    else if (context.canceled)
    {
        isRunning = false;
    }
}

0 references
public void OnJump(InputAction.CallbackContext context)
{
    if (context.started && touchingDirections.isGrounded && canMove)
    {
        animator.SetTrigger("jump");
        rb.velocity = new Vector2(rb.velocity.x, jumpImpulse);
    }
}

0 references
public void OnAttack(InputAction.CallbackContext context)
{
    if (context.performed)
    {
        animator.SetTrigger("attack");
    }
}

0 references
public void OnRangeAttack(InputAction.CallbackContext context)
{
    if (context.performed)
    {
        animator.SetTrigger("rangeAttack");
    }
}

0 references
public void OnHit(int damage, Vector2 knockBack)
{
```

Player Controller

```
3 references
public bool Hit(int damage, Vector2 knockBack)
{
    if (isAlive && !isInvisible)
    {
        Health -= damage;
        isInvisible = true;
        animator.SetTrigger("hit");
        isHit = true;
        damageableHit?.Invoke(damage, knockBack);
        CharacterEvents.characterDamaged.Invoke(gameObject, damage);

        return true;
    }
    return false;
}
```

Damage Component

```
1 reference
public bool Heal(int healthRestore)
{
    if (isAlive && Health < _maxhealth)
    {
        int maxHeal = Mathf.Max(_maxhealth - Health, 0);
        int actualHeal = Mathf.Min(healthRestore, _maxhealth);
        Health += actualHeal;

        CharacterEvents.characterHealed.Invoke(gameObject, healthRestore);
        return true;
    }
    return false;
}
```

Health Component